Deep Recurrent Models for Stock Price Forecasting: A Comprehensive Analysis of Architectures and Hyperparameter Optimization on Google Data

Shashikant Chaudhary
University of Adelaide

Abstract

Accurately forecasting stock prices from historical data is problematic due to nonlinear patterns and long-range temporal dependencies. In this paper, we will compare several architectures of recurrent neural networks RNN, LSTM, GRU, and attention-based LSTM on a large historical dataset of Google stock prices. We use Optuna for hyperparameter optimization to systematically refine model configurations. Contrary to the expectations that more complex architectures would perform best, our results indicate that a properly tuned GRU model achieves the highest accuracy: MSE of 3.0658, RMSE of 1.7509, and R^2 of 0.9708. Even a well-optimized RNN performs very well, underlining the fact that often hyperparameter optimization is way more important than complexity. These findings emphasize the need for adequate model tuning and question whether complexity alone automatically leads to better predictive performance.

1. Introduction

Stock market prediction has attracted considerable attention in both the machine learning and finance communities because of its importance in portfolio management, risk analysis, and algorithmic trading strategies [1]. Traditional time-series forecasting models, such as ARIMA [2] and VAR [3], are limited in their capacity to model nonlinearities and long-range temporal dependencies. Most recently, deep learning techniques, predominantly RNNs, have shown great promise as a new alternative to the conventional approaches, which leverages the ability of RNNs for learning complex temporal patterns from voluminous amounts of data [4, 5]. However, RNN architectures face well-known issues with vanishing and exploding gradients when modeling long sequences [6].

Such models' inefficiency with regards to these problems inspired more enhanced types, including Long Short-Term Memory (LSTM) [7] networks and Gated Recurrent Units (GRUs) [8]. This allows various other improvements in the form of different gates for enabling models to learn over big temporal windows. More recently, further performance gains have come with attention mechanisms [9, 12], where it has become possible to let models concentrate only on such timesteps that would convey maximum information, hence giving greater interpretability and quality in a prediction across many domains such as finance [10].

In this work, we consider an extensive empirical comparison of four recurrent architectures that have been state-of-the-art in stock price forecasting:

- An **RNN** model to start off with.
- An **LSTM-based** model which can capture the long-term dependencies.
- A **GRU-based** model showing computational efficiency.
- An **Attention-based LSTM** model which allows selective focus on critical time steps.

Later, we performed hyperparameter optimization using the Optuna framework [14], a system that systematically searches for the best configuration of the model. We also show that architectural complexity does not always lead to improved performance. By systematic hyperparameter optimization, even simpler models can achieve better performance than their complex counterparts on the given dataset. This work shows in choosing the best predictive models for algorithmic trading and risk assessment for investors and financial analysts.

2. Related Work

Time-series forecasting has undergone great development in recent years, with the arrival of new architectures in the field of deep learning. On the other hand, early approaches relying on feed-forward neural networks or simple RNNs performed poorly when it came to processing sequential dependencies over long time horizons, but modern

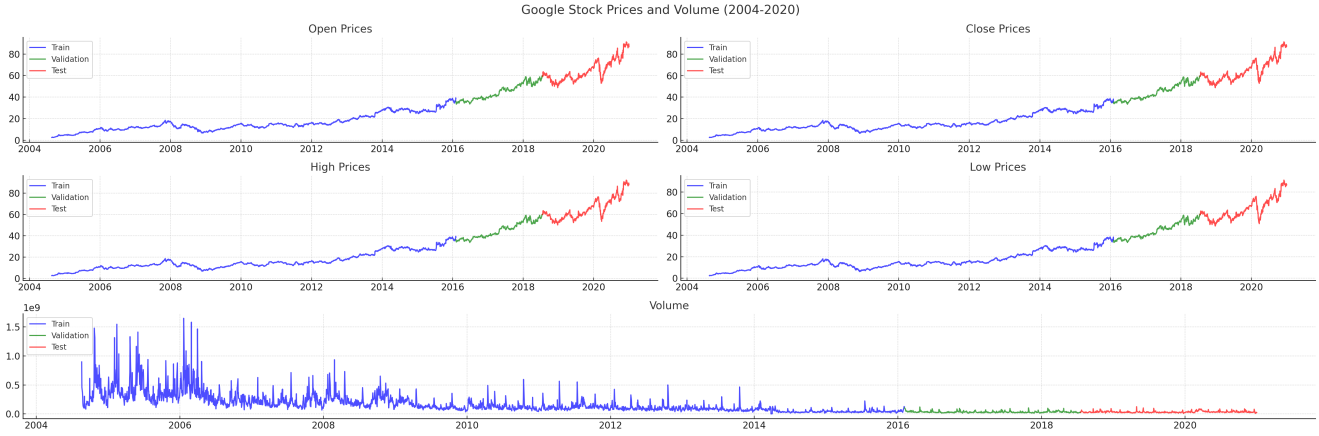


Figure 1. Google stock prices and volume (2004-2020) with data splits.

models, such as Long Short-Term Memory and Gated Recurrent Unit, allow these issues to be overcome and achieve superior performance in predicting tasks related to time-series. [15] [16].

Attention mechanisms were first developed for neural machine translation by Bahdanau et al. and later successfully adapted to time-series forecasting [9]. The added value of these models lies in the dynamic weighting of different time steps with respect to their importance, which has helped in improving predictive performance in energy demand forecasting and traffic prediction tasks significantly [17] [18]. Further, dual-stage and hybrid attention-based models have demonstrated their effectiveness in multivariate settings [10] [11].

Hyperparameter optimization plays a key role in finding the best parameters for the models. An important characteristic of hyperparameter tuning in time-series forecasting is the use of automated frameworks. Optuna has recently gained popularity because it offers more efficiency compared to traditional methods of grid and random search for model parameter optimizations [14]. Studies suggest that, proper tuning yields much better accuracy, especially on challenging applications related to financial and stock market predictions [13].

While the statistical techniques developed earlier, such as ARIMA and VAR, work well in modeling time-series data which are linear and stationarity. However it cannot deal properly with handling nonlinear and multivariate dependencies which makes the deep learning approach indispensable for modern forecasting problems [16] [18].

3. Methodology

3.1. Dataset and Modeling Strategy

The historical daily stock price of Google(GOOG) used in this paper ranges from August 2004 to December 2020. Such a period encompasses different market scenarios, from relative stability to highly volatile situations which provides perfect conditions to build a robust and accurate model. The method which is being used here focuses on the prediction of one step ahead forecasting after inputting information from the previous window of 30 days.

3.1.1 Dataset Overview

The dataset has been visualized in Fig. 1 and contains the following attributes mentioned below:

- **Date:** Day, when trading was done.
- **Open:** Price when trading started.
- **High:** Highest price during the trading session.
- **Low:** Lowest price during the trading session.
- **Close:** Price at the end of the trading session(forecast target).
- **Volume:**Total number shares traded on that session(date).
- **Adjusted Close:** Closing price which is adjusted for corporate actions such as stock splits and dividends.

Date	Open	High	Low	Close	Volume	Adj. Close
20-Nov-08	6.85	7.05	6.45	6.46	392.64M	6.46
22-Jul-15	33.04	33.93	32.95	33.11	78.59M	33.11
03-Feb-10	13.17	13.50	13.16	13.47	120.41M	13.47
10-Jun-22	112.78	113.50	110.86	111.43	31.32M	111.43
28-Oct-19	63.77	64.97	63.63	64.50	52.26M	64.50

Table 1. Sample Records from the Google Dataset

3.1.2 Modeling Strategy

The modeling approach will emphasize the forecast of the closing price one trading day ahead after a 30-day historical observation window. The window of 30 days agrees

Aspect	Choice
Data Range	Aug 2004 – Dec 2020
Target Variable	Closing Price
Input Sequence	30 consecutive trading days
Forecast Horizon	1 day ahead

Table 2. Key Modeling Choices

with a usual trading month; hence, it's kind of a natural choice if one wants to highlight the recent market behaviors and tendencies without adding unnecessary complications. The 30-day window provides a good balance in including relevant information without significant risks of overfitting. Key modeling decisions are presented in Table 2.

Rationale:

- **Data Range:** Using the data from 2004 up to 2020 diversifies market conditions. In fact, the chosen endpoint comes before significant anomalies caused by the recent pandemic; therefore this lets the model focus on more general market behaviors.
- **Target Variable:** The closing price is stable and one of the most analyzed measures, less prone to intra-day fluctuations, and hence more reliable for end-of-day predictions.
- **Input Sequence:** The 30-day historical window will capture the short-term trends and patterns. This provides a good balance in including relevant information without significant risks of overfitting.
- **Forecast Horizon:** One-day-ahead forecasting is practical and common; hence, it allows the predictions after the 30-day window in such a way that it is actionable without complicated multi-step forecasts.

In sum, the strategy has ensured that the model is based on firm historical data and a straightforward forecasting horizon that can offer realistic and reliable predictions of stock prices for effective decision-making.

3.2. Data Split and Preprocessing

The data is split into train(70%), validation(15%), and test(15%) and was scaled using MinMax scaler [0,1] for stable training.

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{1}$$

4. Model Architectures

We study four RNN architectures for time series forecasting: RNN, LSTM, GRU, and attention-based LSTM. All these models take as input a window of 30 historic stock prices to predict the price for the following day. Figure 2 visualizes the RNN, LSTM, and GRU structures.

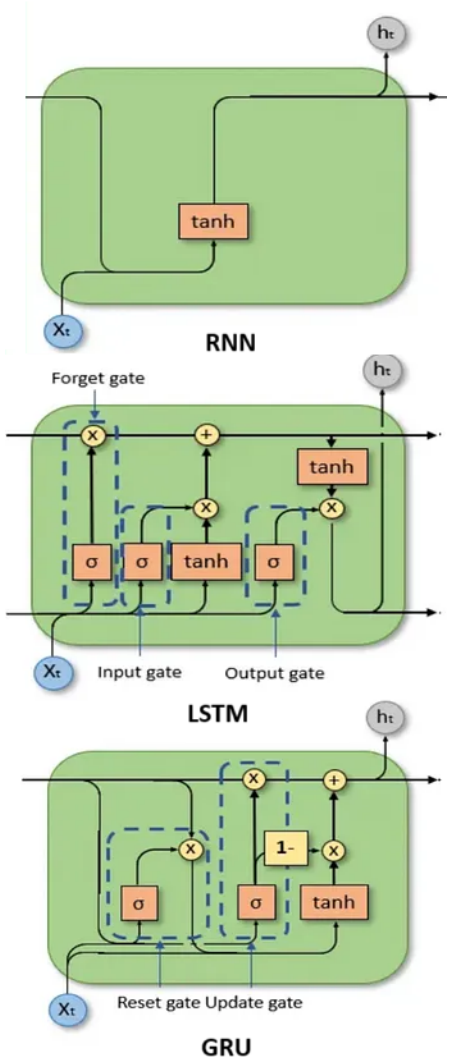


Figure 2. Comparison of RNN, LSTM, and GRU: x_t is input, h_t is hidden state, with LSTM adding cell state and gates, and GRU using reset and update gates [19].

4.1. RNN

The RNN consists of a simple structure where the hidden state at each time step h_t is updated based on the previous hidden state, h_{t-1}, and the current input x_t. It is defined mathematically as:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h), \tag{2}$$

$$y_t = W_{hy}h_t + b_y, \tag{3}$$

where x_t is the input at time t, h_t is the hidden state, and y_t is the output. The model parameters include:

- W_{hh}, W_{xh}, W_{hy}: Weight matrices,
- b_h, b_y: Biases.

While RNNs can capture the relationships that come out in a series, vanishing gradients make it hard to capture the long-term dependencies in them [6]; therefore, they remain difficult for challenging tasks involving time series.

4.2. LSTM

The Long Short-Term Memory network overcomes the problems present in RNN by introducing the concept of a gate that regulates the flow of information. This helps the network to retain long-term dependencies effectively [7]. The LSTM components include:

- **Forget Gate (f_t):** This gate is responsible for deciding which information to be discarded.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \tag{4}$$

- **Input Gate (i_t):** This gate is responsible for deciding which new information to be stored.

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \tag{5}$$
$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c), \tag{6}$$

- **Cell State Update:** Combines the forget and input gates to update the cell state.

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \tag{7}$$

- **Output Gate (o_t):** It determines the output based on the updated cell state.

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \tag{8}$$
$$h_t = o_t \odot \tanh(c_t). \tag{9}$$

Here, f_t, i_t, o_t denote the forget, input, and output gates, respectively; c_t is the cell state; and h_t is the hidden state. $\odot$ represents element-wise multiplication. The weights W_f, W_i, W_c, W_o and biases b_f, b_i, b_c, b_o are learnable parameters. LSTMs perform exceptionally well in learning long-term temporal patterns and have been a popular choice for time series forecasting.

4.3. GRU

The Gated Recurrent Unit or GRU simplifies the above LSTM architecture but has similar performance. GRUs use fewer gates and also merge the forget and input gates into a single update gate [8]. The components of the model are:

- **Update Gate (z_t):** The gate decides how much of the past information to retain.

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z), \tag{10}$$

- **Reset Gate (r_t):** Controls how much the model is influenced by the previously hidden state.

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r), \tag{11}$$

- **Candidate Hidden State:** Computes the new hidden state based on the reset gate.

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h), \tag{12}$$

- **Hidden State Update:**

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \tag{13}$$

Here, z_t and r_t denote the update and reset gates, respectively; $\tilde{h}_t$ represents the candidate hidden state, while h_t denotes the updated hidden state. Actually, the GRUs are computationally efficient and suitable for those datasets where the size of the dataset is limited.

4.4. Attention-based LSTM

Attention-based LSTM improves on the original by allowing an LSTM network to selectively concentrate on parts of the input sequence that matter most. This is achieved via the attention mechanism which is involved in assigning weights to hidden states [10]. The process is as follows:

- **Score Computation:** Calculates the importance of each hidden state h_t.

$$e_t = v^\top \tanh(W_a h_t), \tag{14}$$

- **Attention Weights:** Normalizes the scores to obtain weights.

$$\alpha_t = \frac{\exp(e_t)}{\sum_k \exp(e_k)}, \tag{15}$$

- **Context Vector:** Computes a weighted sum of hidden states.

$$c = \sum_t \alpha_t h_t. \tag{16}$$

This enables the model to selectively focus on the time instant of interest. Such a context vector c aids interpretability and improves predictive performance.

5. Code: Public Link

https://github.com/chaudharysk/DeepL-Assingment3

6. Experiments and Results

The code for the experiments was implemented in Python by using the PyTorch framework and the best parameters were obtained from the optuna framework.

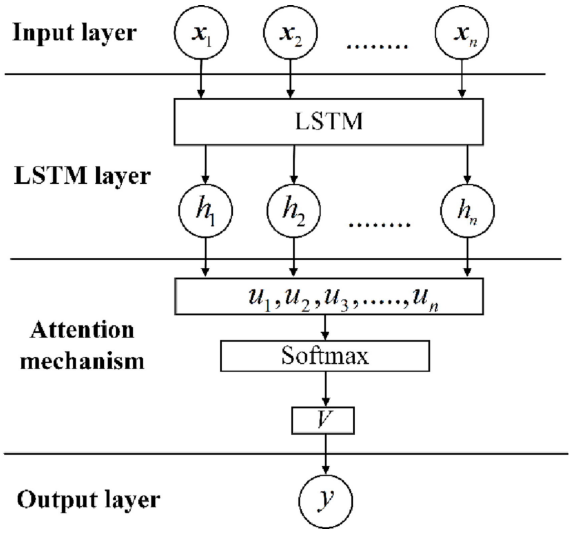


Figure 3. Architecture of Attention-based LSTM [20].

6.1. Best Hyperparameters for Models

Table 3 lists the best hyperparameters found for each architecture after 25 Optuna trials per model.

Model	Hidden Size	Num Layers	Learning Rate	Batch Size
RNN	128	1	0.0043	16
LSTM	98	1	0.0050	32
GRU	99	1	0.0014	16
Attention-LSTM	45	1	0.0044	16

Table 3. Best Hyperparameters Found via Optuna

6.2. Evaluation Metrics

For the models implemented, we use the following metrics while evaluating the performance of our models on the test sets:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2, \tag{17}$$

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|, \tag{18}$$

$$\text{RMSE} = \sqrt{\text{MSE}}, \tag{19}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2}. \tag{20}$$

Here, y_i represents the actual value for the i-th data point, $\hat{y}_i$ is the predicted value for the i-th data point, $\bar{y}$ denotes the mean of the actual values, calculated as $\bar{y} = \frac{1}{N}\sum_{i=1}^{N} y_i$, and N is the total number of data points in the test set.

6.3. Quantitative Results

A simple RNN model with a hidden size of 16, 1 layer, Adam optimizer, and learning rate 0.001 was implemented as the baseline before applying any optimization or hyperparameter tuning. The results for this model are as follows:

MSE	MAE	RMSE	R^2
93.5462	8.1362	9.6719	0.1082

Table 4. Performance Metrics

This baseline puts into light how important proper architecture and hyperparameter tuning are. Further experiments with the best hyperparameters found by Optuna have shown large improvements.

Model	MSE	MAE	RMSE	R^2
RNN	13.8485	3.1385	3.7214	0.8680
LSTM	6.8092	2.1640	2.6094	0.9351
GRU	**3.0658**	**1.3171**	**1.7509**	**0.9708**
Attention-based LSTM	7.2091	2.0582	2.6850	0.9313

Table 5. Test Set Performance Metrics

Table 5 clearly shows that GRU model shows its overall best performance due to its efficient gating mechanism, while LSTM is still competitive due to its strong focus on capturing temporal dependencies.

6.4. Training and Validation Curves

Figure 5 shows loss curves of training and validation of the GRU model. After some epochs, it can be observed that the value of validation loss converges to a low value indicating better generalization. Early stopping also helps in halting training before it overfits.

6.5. Predictions vs. Actual Values

Figure 6 shows the graph of actual vs. predicted prices on the test dataset of the GRU model. It is evident from the picture that it closely follows the true trend whether it is an upward or downward trend.

7. Discussion

Our results generally confirm the tendencies in related literature [4, 5, 10], where better performance is shown by more advanced architectures. However, our results also challenge the assumption that increased architectural complexity necessarily yields improved results, being a simpler model GRU outperformed the attention-based LSTM architecture.

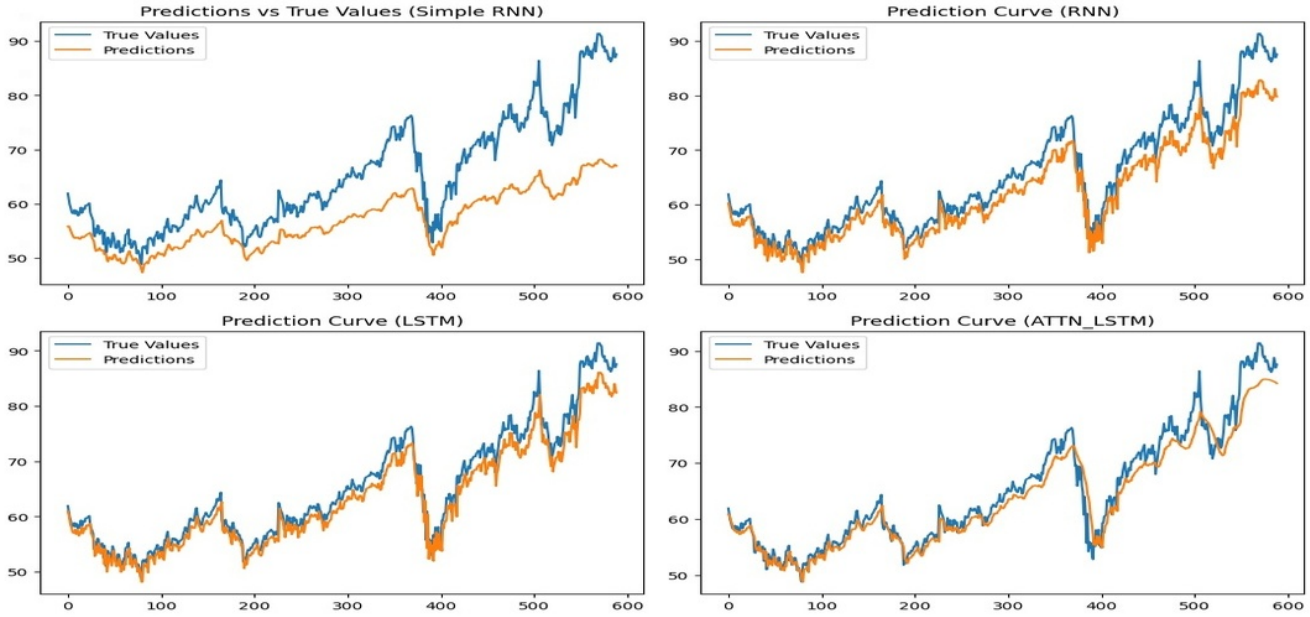


Figure 4. Actual vs Prediction curve on the test set of different models.

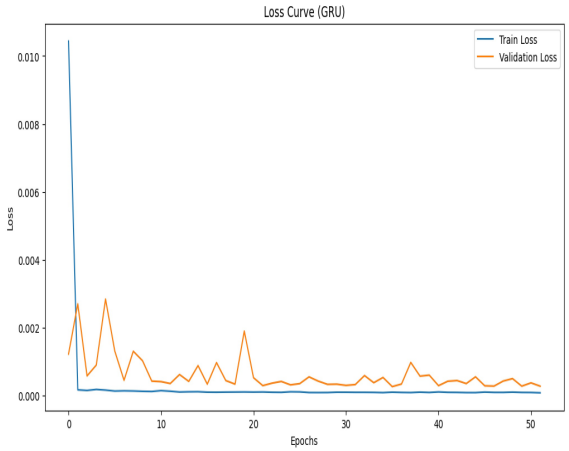


Figure 5. Training and Validation Loss Curves for the GRU.

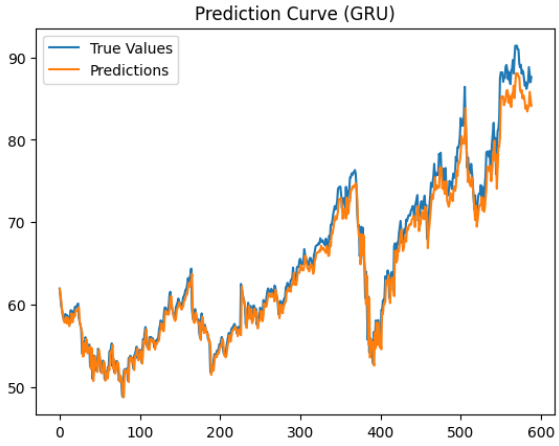


Figure 6. Actual vs Prediction curve of the GRU on the test set.

Without the optimization, the RNN(hidden size: 16, 1 layer, learning rate: 0.001) baseline yielded poor performance, with an MSE of **93.5462**, MAE of 8.1362, and R^2 of 0.1082.These metrics thus point out the fact that this model completely failed in learning the temporal dependencies in this data, and further supports that careful architecture and hyperparameter tuning are required for improved results.

After systematic optimization, the overall best performance of the GRU model(MSE of **3.0658**, MAE of 1.3171, $RMSE$ of 1.7509, and $R^2$of 0.9708) indicates that its efficient gating mechanism suits this dataset most. This means that in the case of datasets with relatively small input windows and localized dependencies, GRUs can provide the best balance between complexity and accuracy.

The Attention-based LSTMs speak to the strong performance(MSE of 7.2091) of the selective weighting of temporal information. It again underlines the relevance of tuning model architecture with respect to inherent structure and dependabilities from the problem at stake. However, as those performances did not really outperform GRUs, that might hint at the temporal characteristics of the data and, eventually, of the input window, which was not long enough to allow the attention mechanism to perform up to its strengths. It again underlines the relevance of tuning model architecture with respect to inherent structure and de-

pendabilities from the problem at stake.

By using Optuna for hyperparameter optimization, gave further improvements beyond naive choices of parameters, highlighting again that proper tuning is highly important in real-world applications [14].

Nevertheless, no model can be perfect because of the market noise, exogenous events, and unobservable variables that cannot be modeled accurately. By including additional features, such as sentiment scores, macroeconomic indicators, or the application of transformer-based architectures [12], can further improve these results.

8. Conclusion

In this paper, different types of recurrent neural networks RNN, LSTM, GRU, and attention-based LSTM have been experimented upon for stock price forecasting of Google stock prices. We have used Optuna framework for hyperparameter optimization because it is considered quite efficient in doing a fast and unbiased search among available hyperparameter combinations. Using these methods reveals that proper tuning may create greater differences in performance than just model complexity. Among these, the GRU model was the best option with the lowest error rate and highest predictive accuracy. Moreover, the RNN performed better than the LSTM and attention-based LSTM models after appropriate tuning, proving the fact that simplicity enforces results if optimized well.

These findings challenge the assumption that more advanced architectures are inherently better performers and instead bring to the forefront how hyperparameter optimization is important in a real-world setup. The balance of the GRU between computational efficiency and performance, and the interpretability for understanding the temporal importance using the Attention-based LSTM make them very promising models for financial time series forecasting.

The work developed herein therefore provides a good insight, into which future research can build by adding more feature dimensions, such as including macroeconomic data or sentiment analysis, and more sophisticated architectures, possibly including transformers.

9. Future Work

Future work can focus on the following:

- Rich context - more features include news sentiments and macroeconomic data.
- Investigation into transformer-based models that would do even better in long-term temporal modeling.
- Multi-step ahead predictors as forecasting horizon.
- Uncertainty Quantification and probabilistic predictions for robust risk management strategies.
- Investigating the application of transfer learning for the problem of learning from data about several related financial instruments.
- Design hybrid models that embed deep learning into traditional statistical approaches to enhance robustness.
- Study the impact of the selected feature selection and feature engineering methods on model interpretability and performance.

The paper is a deep methodological analysis that might serve as a starting point for any researcher or enthusiast in the construction of interpretable and efficient deep-learning models on financial time series forecasting.

References

[1] E. F. Fama, "Efficient Capital Markets: A Review of Theory and Empirical Work," *The Journal of Finance*, vol. 25, no. 2, pp. 383–417, 1970. doi: 10.2307/2325486.

[2] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*. Wiley, 2015. ISBN: 9781118674925.

[3] H. Lütkepohl, *New Introduction to Multiple Time Series Analysis*. 1st ed. Springer-Verlag Berlin Heidelberg, 2005, pp. XXI, 764. doi: 10.1007/978-3-540-27752-1.

[4] M. Hiransha, E. Abraham, K. P. Vinayakumar, and S. Soman, "NSE Stock Market Prediction Using Deep-Learning Models," *Procedia Computer Science*, vol. 132, pp. 1351–1362, 2017. doi: 10.1016/j.procs.2018.05.050.

[5] T. Fischer and C. Krauss, "Deep Learning with Long Short-term Memory Networks for Financial Market Predictions," *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, 2018. doi: 10.1016/j.ejor.2017.11.054.

[6] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, Mar. 1994. doi: 10.1109/72.279181.

[7] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997. doi: 10.1162/neco.1997.9.8.1735.

[8] K. Cho, B. van Merriënboer, Ç. Gülçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning Phrase Representations Using RNN

Encoder-Decoder for Statistical Machine Translation," *ArXiv*, 2014.

[9] D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," *ArXiv*, 2014.

[10] Y. Qin, D. Song, H. Chen, W. Cheng, G. Jiang, and G. W. Cottrell, "A Dual-Stage Attention-Based Recurrent Neural Network for Time Series Prediction," in *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI-17)*, 2017, pp. 2627–2633. doi: 10.24963/ijcai.2017/366.

[11] S. Sang and L. Li, "A Novel Variant of LSTM Stock Prediction Method Incorporating Attention Mechanism," *Mathematics*, vol. 12, no. 7, p. 945, 2023. doi: 10.3390/math12070945.

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, Curran Associates, Inc., 2017.

[13] E. Ismanto and V. Vitriani, "LSTM Network Hyperparameter Optimization for Stock Price Prediction Using the Optuna Framework," *Jurnal Ilmiah Teknik Elektro Komputer dan Informatika (JITEKI)*, vol. 9, no. 1, pp. 22–35, 2023.

[14] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," *ArXiv*, 2019.

[15] Y. Liu, S. Zheng, C. Feng, C. Chen, and J. Zou, "DSTP-RNN: A Dual-stage Two-phase Attention-based Recurrent Neural Networks for Long-term and Multivariate Time Series Prediction," *ArXiv*, 2019.

[16] Z. Niu, H. Liu, and Y. Zhang, "Wind Power Forecasting Using Attention-based Gated Recurrent Unit Network," *Energy*, vol. 196, p. 117081, 2020. doi: 10.1016/j.energy.2020.117081.

[17] Q. Tan, "DATA-GRU: Dual-Attention Time-Aware Gated Recurrent Unit for Irregular Multivariate Time Series," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 1, pp. 930–937, Apr. 2020.

[18] S. Zhou, F. Wu, Y. Yang, J. Zhang, and Y. Zhang, "A Hybrid Framework for Multivariate Time Series Forecasting of Daily Urban Water Demand Using Attention-Based Convolutional Neural Network and Long Short-Term Memory Network," *Sustainability*, vol. 14, no. 17, p. 11086, 2021. doi: 10.3390/su141711086.

[19] J. Dabass, "Introducing GRU, RNN, and LSTM: A Beginner's Guide to Understanding These Revolutionary Deep Learning Models," Python in Plain English, May 21, 2024. [Online]. Available: https://python.plainenglish.io/introducing-gru-rnn-and-lstm-a-beginners-guide-to-understanding-these-revolutionary-deep-35b509a34a5a

[20] Q. Kang, X. Liu, and J. Zhang, "Attention-based LSTM Predictive Model for the Attitude and Position of Shield Machine in Tunneling," *Underground Space*, vol. 13, pp. 335–350, 2023. doi: 10.1016/j.undsp.2023.05.006.